Implementation of a NN Regressor through the Edge Impulse Platform

Matthew Sylvester

February 2026

Contents

1 Legal information

As of the date on this document, code output from the Edge Impulse platform developer (free) tier uses an Apache 2.0 license (see in comment of code output) and can be used commercially without restriction.

2 Tools required

In order to follow the process described by this document, you must have the following tools:

- The Arduino IDE
- The requisite Arduino library and board manager for deploying code to your microcontroller architecture of choice
- The requisite USB drivers for your microcontroller of choice
- A "developer tier" Edge Impulse account
- A dataset to train your model on, or a trained TensorFlow model

3 Process Justification & Notes

Why do we use edge impulse? It provides a dataset to model to library workflow. Of particular use is the model to library portion, for easy deployment to the edge via the simple workable format of an Arduino library.

Why do we use the Arduino IDE / Arduino library as deployment tools? Using the Arduino IDE & library format is the easiest way to get code onto a real microcontroller without setting up a complex development environment yourself. It handles the flashing process using AVRDUDE in the background, and comes prepackaged with tools to automatically handle building your code for a specific architecture.

After testing building a few models using the built-in model creator, would be best to migrate model creation and testing to a python notebook in order to perform complex tuning and testing in a more customizable environment. Once at that stage, Edge Impulse will be in the workflow as a way to package a model into a callable library for deployment.

4 Procedure

4.1 Create a model in Edge Impulse

If using a pre-trained Tensorflow model, skip this step.

On your edge impulse profile page, create a new project. Then, open the data acquisition page and run the CSV Wizard to configure data ingestion for the type of data you will be using. For the 16 sensor dataset provided by Jibril, you can alternatively upload a .json file with the following configuration, while making sure that the filename you have uploaded matches the .json configuration:

```
{
    "version": 1,
    "fileName": "ethylene_methane_ds_10hz.parquet",
    "created": 1771281327865,
    "delimiter": ",",
    "skipFirstLines": 0,
    "spec": {
        "type": "timeseries-row",
        "labelsColumn": "methane_ppm",
        "valueColumns": [
            "s01",
            "s02",
            "s03",
            "s04",
```

```
            "s05",
            "s06",
            "s07",
            "s08",
            "s09",
            "s10",
            "s11",
            "s12",
            "s13",
            "s14",
            "s15",
            "s16"
        ],
        "timestamp": {
            "type": "column",
            "timestampColumn": "time_s",
            "format": "elapsed-seconds",
            "frequency": 10
        },
        "limitSampleLengthMs": 100000,
        "dealWithMultipleLabels": "use-last-label"
    }
}
```

Note that this configuration splits the data into 100 second intervals. Modify the limitSampleLengthMs key if you wish to sample for a different time interval.

Upload the dataset. The data will automatically be split into testing and training sets. Click on the "Create Impulse" section. You'll be prompted to choose your target device. For the ESP's we have been using, use the following configuration:

- Target device: Custom
- Processor family: ESP32
- Clock rate: 240MHZ
- Accelerator: None
- Custom device name: ESP
- RAM: 16MB
- ROM: 1MB
- Latency: 100ms

Add a raw data processing block with S01 to S16 as input features. Add a regression learning block and save the impulse.

Under the Raw Data section, click Generate features. Generate your features with scikit-learn StandardScalar enabled to ensure the input signals are weighted equally.

Under the Regression section, pick your neural network settings. Save and train the model.

5 Deploy the model

Under Deployment, select Arduino library as the deployment target. Build the model, and download it to your device.

Follow the EI docs for instructions on how to load the Arduino library into the Arduino IDE.

Make sure that you have the correct drivers to communicate with the microcontroller you are using. For the ESP we have, this is the CH343SER USB to serial communications module.

Make sure you have set the microcontroller configuration correctly in the IDE. For the ESP, first go to the Boards Manager on the sidebar, then type in "esp", and add the board manager made by Espressif Systems. Then under the Tools menu, go to Board, esp32, and select ESP32 Dev Module. do the following under the Tools menu:

- CPU Frequency: 240MHz
- Flash size: 16MB
- Partition scheme: 16MB flash with 3MB app
- PSRAM: Disabled

All other options should remain on default. Plug in the microcontroller, and in the dropdown menu on the top of the IDE, pick the COM port that corresponds to the controller you just plugged in (unplugging and plugging it back in can make it obvious which port to pick). Pick ESP32 DevModule as the board in the popup.

From here, you can open the "static buffer" example sketch under File > examples > your-library-name > static buffer and upload it to the microcontroller. Make sure to populate the input_buffer variable and instantiate it with as many values as there are input features in the model, which will be 140 if you use the config provided.